

TECNOLOGÍAS DE LA INFORMACIÓN

Desarrollo de Sistemas Informáticos

4 créditos



Profesores:

Ing. Jorge Arun Zambrano Cedeño, Mg.

Titulaciones	Semestre
• <i>Tecnologías de la Información en Línea</i>	<i>Quinto</i>

NOMBRES Y APELLIDOS:

GERSON DAVID GUACHO UZHO

PARALELO:

A

SEMESTRE:

QUINTO

ACTIVIDAD:

Actividad #1: Estudio de Caso - Sistema de Gestión de Incidentes (Help Desk)

PERIODO ABRIL 2026 - AGOSTO 2026



Actividades a desarrollar en la Unidad 1

Actividad # 1: Estudio de caso - Sistema de Gestión de Incidentes (Help Desk)

Objetivo de la tarea:

- Diseñar la arquitectura lógica de un sistema de gestión de incidentes técnicos (Help Desk) para infraestructura de servidores que permita aplicar correctamente patrones de diseño de software y flujos de trabajo profesionales (Gitflow), fortaleciendo la capacidad de análisis, abstracción y buenas prácticas en el desarrollo de soluciones escalables, reutilizables y mantenibles..

Disponibilidad: lunes, 20 de abril de 2026, 00:00

Fecha límite: sábado, 16 de mayo de 2026, 23:59

Descripción de la Actividad

- El estudiante deberá diseñar un sistema de Help Desk orientado a la gestión de fallos en un Data Center. El sistema debe permitir reportar incidentes (hardware, red, software), asignar técnicos especializados y gestionar estados de resolución. Para lograr una arquitectura clara y profesional, se deberá utilizar obligatoriamente un repositorio en GitHub bajo el modelo Gitflow y aplicar al menos tres patrones de diseño de software, justificando su elección y uso técnico.

Calificación

- Tendrá una ponderación de 10 puntos.

Instrucciones

- **Análisis del problema**
 - Identificar los actores principales (Usuario final, Técnico de soporte, Administrador de sistemas).
 - Definir los requerimientos funcionales (Crear ticket, categorizar fallo, asignar técnico) y no funcionales (Escalabilidad, mantenibilidad, trazabilidad de versiones).
- **Diseño del sistema**



- Crear un repositorio público en GitHub configurando la estructura de ramas Gitflow (master, develop y ramas feature/ para avances).
- Crear un diagrama de casos de uso para entender las interacciones del personal de TI con la infraestructura.
- Realizar un diagrama de clases UML donde se evidencien los objetos y su relación bajo principios SOLID.
- Diseñar la arquitectura general del sistema integrando el flujo de versiones.
- **Aplicación de patrones de diseño**

Utilizar al menos tres patrones. Algunos recomendados:

 - Singleton: Para la instancia central del motor de gestión de tickets y conexión a datos.
 - Observer: Indispensable para el sistema de notificaciones automáticas a técnicos ante nuevos incidentes (Base de la programación reactiva).
 - Factory Method: Para la creación de diferentes tipos de incidentes según el área (Red, Hardware, Software).
 - MVC (Model-View-Controller): Como patrón estructural para organizar la lógica de negocio y la interfaz.
- **Documentación del diseño**
 - Explicar el propósito de cada patrón utilizado y cómo resuelve un problema específico del Help Desk.
 - Justificar el uso de Gitflow: Incluir el enlace al repositorio de GitHub y capturas de pantalla del historial de ramas/commits que evidencien el flujo de trabajo.
 - Incluir diagramas de clases y secuencia donde se refleje la implementación de los patrones.

Recomendaciones

- Presentar la tarea con un adecuado formato y presentación de régimen universitario en archivo pdf. Utilizar el formato



"apellido_nombre_actividad1.pdf", estará en la sección de Materiales de aprendizaje / Recursos - Unidad 1 / Recursos Complementarios de la Unidad 1.

- La actividad deberá ser desarrollada en el tiempo estipulado.
- La actividad es individual y por lo tanto cada estudiante asumirá la responsabilidad de su autoría.
- Asegúrate de nombrar y justificar claramente cada patrón.
- Revisa que los diagramas sean legibles y coherentes con el diseño planteado.
- Usa nombres significativos para las clases y métodos.
- No olvides cuidar la presentación del documento: ortografía, redacción clara, numeración de secciones.
- Puedes usar herramientas como Draw.io, Lucidchart o StarUML para los diagramas.



Desarrollo

Actores principales

- **Usuario final:** Es el usuario encargado de reportar el fallo mediante la creación de un ticket que contiene la descripción, evidencia y una categorización preliminar del fallo ya sea de RED, SOFTWARE o HARDWARE.
- **Técnico de soporte:** Es el usuario que está encargado de atender los tickets que le asigne el administrador o el sistema. Cambia los estados del ticket y da por resuelto el ticket.
- **Administrador de sistemas:** Es la persona encargada de validar la categorización preliminar que dio el usuario al problema y después se encarga de asignar un técnico para dicho fallo

Requerimientos funcionales

Número	Requerimiento	Descripción	Prioridad
RF1	Crear ticket	El usuario final crea un ticket del error, ingresando la categorización del fallo, evidencia y una breve descripción.	Alta
RF2	Categorizar fallo	El usuario final hará una categorización preliminar del fallo ya sea de RED, HARDWARE o SOFTWARE	Alta
RF3	Confirmar categorización	El administrador será el encargado de confirmar la categorización que ha asignado el usuario final	Alta
RF4	Asignar Técnico	El administrador después de confirmar la categorización será el encargado de asignar un técnico a dicho problema.	Alta
RF5	Cambiar estado del ticket	El administrador y el técnico serán los encargados de cambiar el estado del ticket	Alta
RF6	Confirmar solución	El usuario final dará por satisfactorio la resolución del ticket	Alta
RF7	Informar de los	El sistema informará a los técnicos de	Alta



	tickets a los técnicos	los nuevos tickets ingresados.	
RF8	Informar del estado de los tickets	El sistema será el encargado de informar el estado de los tickets	Alta
RF9	Registro de usuarios	El software debe disponer de registros de usuarios	Alta
RF9	Login	El sistema tendrá de un sistema de logeo	Alta
RF10	Visualización de los tickets	Contendrá una opción que permita a los usuarios visualizar los tickets.	Alta

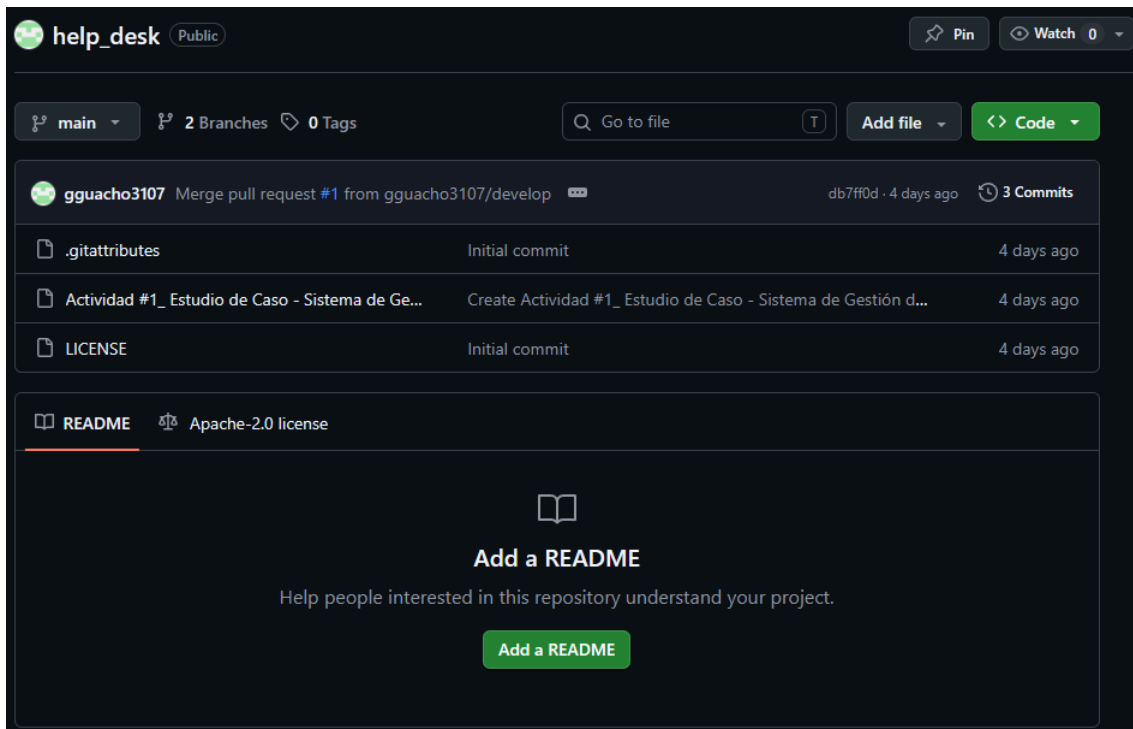
Requerimientos no funcionales

Número	Requerimiento	Descripción	Prioridad
RF1	Mantenibilidad	El software permitirá la corrección de errores, mejorar el rendimiento y adaptarse a nuevas funcionalidades de una manera rápida, segura y económica	Alta
RF2	Escalabilidad	El sistema deberá permitir aumentar su capacidad de respuesta, rendimiento, almacenamiento y procesamiento a medida que crezca los usuarios y los tickets	Alta
RF3	Trazabilidad de versiones	El ciclo del desarrollo del software debe ser auditable y seguir bajo el modelo Gitflow	Alta
RF4	Rendimiento	El sistema debe ser capaz de notificar a los usuarios finales respecto al estado de sus tickets y a los técnicos de los tickets generados en un tiempo no mayor a un segundo.	Alta
RF5	Seguridad e Integridad	El sistema debe tener un control de acceso basado en roles.	Alta
RF6	Disponibilidad	El software debe estar disponible en un	Alta

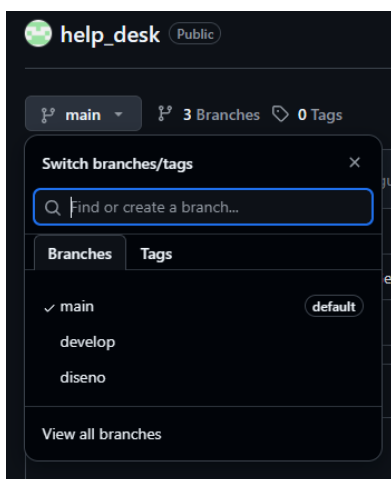


		99.9% del año	
--	--	---------------	--

Crear un repositorio público en GitHub



Configurar las estructuras de ramas



Diagramas de casos de uso



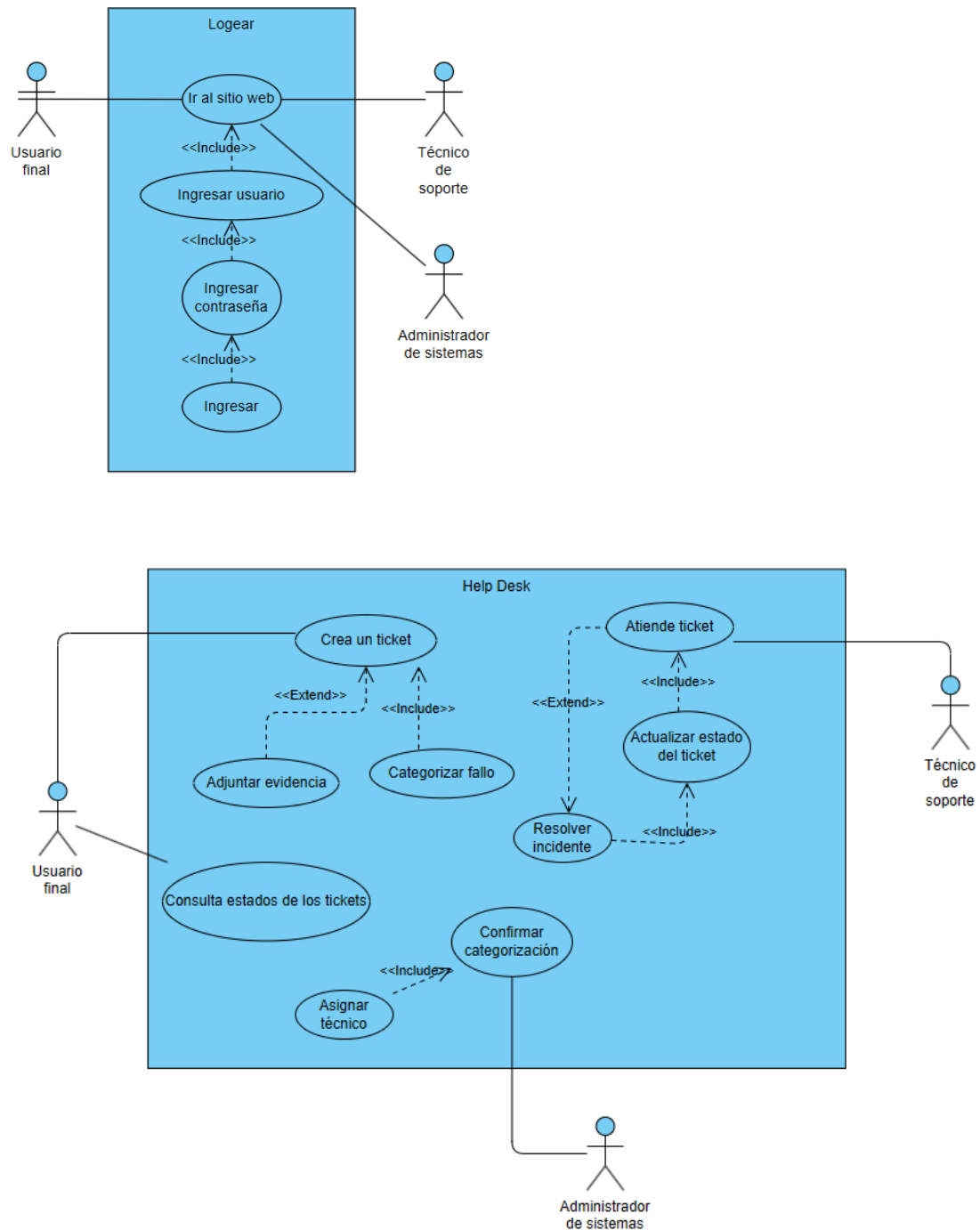
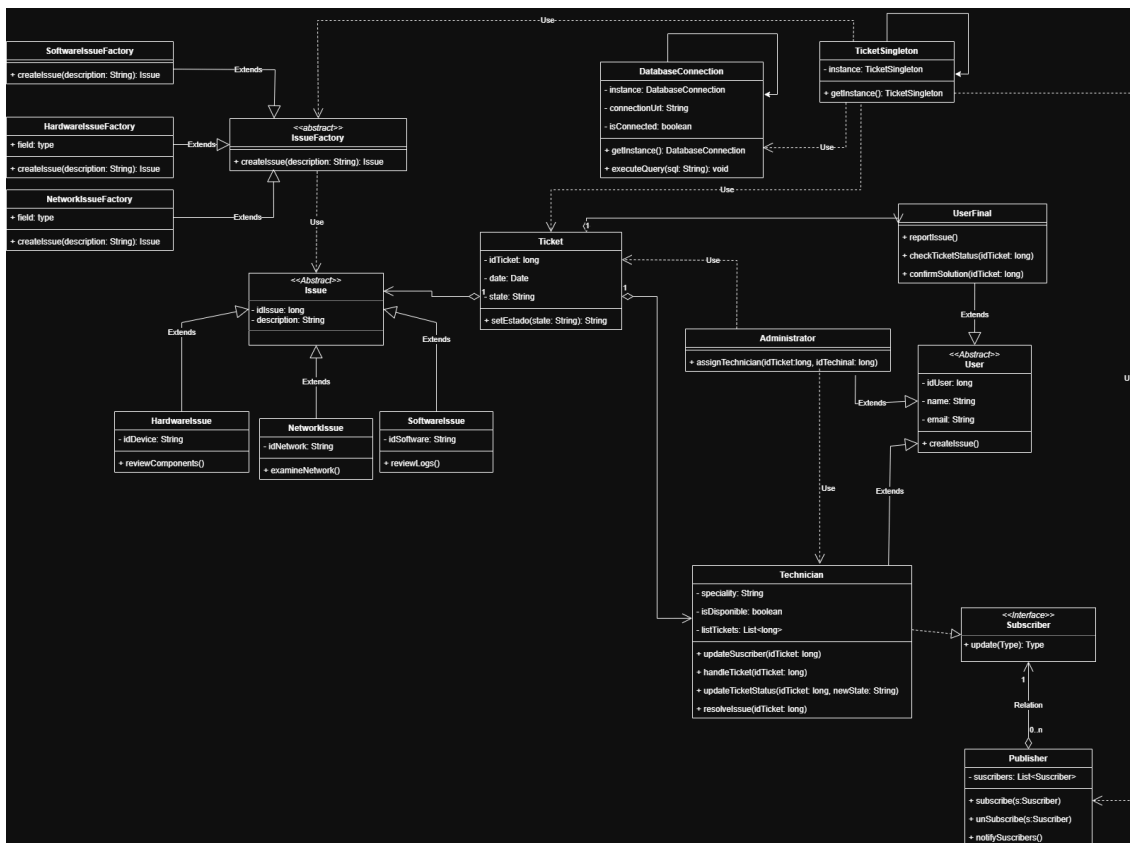


Diagrama de clases UML donde se evidencien los objetos y su relación bajo principios SOLID.



Elección de patrones de diseño

1. **Singleton:** Con este patrón nos aseguramos que nuestro motor de gestión de Tickets y el acceso a los datos tengan un punto de acceso global
2. **Observer:** Soluciona el problema de que el sistema se encargue de notificar a los técnicos de las nuevas solicitudes de tickets y los usuarios podrán estar al tanto de los nuevos estados de los tickets.
3. **Factory Method:** Este patrón nos permite crear diferentes tipos de issues y en un futuro poder implementar más tipos de issues. Lo cual nos permitirá ser escalables y mantenibles.
4. **MVC (Model-View-Controller):** El patrón nos brinda la forma para separar la lógica de negocios y la interfaz.

Uso de Gitflow

Los beneficios de usar Gitflow son muchos y entre ellos están:

1. Desarrollar una nueva funcionalidad en una rama aislada.



2. El uso de ramas divide el ciclo de vida del software en la rama main, develop y esta última se crean más ramas que se fusionan con la develop y luego con la main.
3. El historial nos permite auditar el sistema y comprobar que estamos cumpliendo con nuestro trabajo y plazos de entrega.
4. Facilita la concurrencia ya que permite que varias personas trabajen en un mismo proyecto pero en diferentes funcionalidades, pero en este caso solo estare yo trabajando.

Evidencias

https://github.com/gguacho3107/help_desk/settings